Kawn AI Content Engine
Mobile Developer Integration Guide

How to get AI-generated, theme-based posts into Kawn community feeds automatically.

Kawn Platform

The simple version

The Content Engine is a separate system. It knows each community's theme (football, cricket, AI, etc.), reads related news, writes posts with AI, checks them for safety, and when ready sends them to the Kawn app through an API that you build.

The Flutter app does NOT talk to the Content Engine directly. Your developer builds one API on the Kawn backend. The Content Engine calls it when a post is ready.

Content Engine = the writer | Kawn backend = the door into the app | Flutter app = shows posts normally

What 'theme wise' means

You do not need to write AI logic in the app. When a community is registered in the Content Engine, send its theme info. The engine uses it to generate posts that fit each community.

Theme fields to send

- category - main topic (e.g. football)
- tags - specific theme (e.g. france, les-bleus)
- blocked_topics - topics to avoid (e.g. cricket)
- country - local relevance (e.g. France)
- preferred_tone - how posts should sound (e.g. passionate)
- language - post language (e.g. en or fr)

France Fans gets France football content. India Cricket gets cricket content. And so on.

Post style by time of day (UTC)

- Morning (~8am): morning update
- Midday (~11am): news discussion
- Afternoon (~2pm): poll
- Evening (~6pm): news discussion
- Night (~9pm): evening recap

Your job in 3 steps

Step 1 - Register every Kawn community in the Content Engine

When a community is created or updated in Kawn, the backend should also tell the Content Engine.

Call:

```
POST https://YOUR-CONTENT-ENGINE.onrender.com/api/communities
```

Example body:

```
{
  "name": "France National Team Fans",
  "description": "Fans discussing Les Bleus",
  "category": "football",
  "tags": ["france", "football", "national-team"],
  "blocked_topics": ["cricket"],
  "country": "France",
```

```
    "preferred_tone": "passionate",
    "language": "en",
    "kawn_community_id": "12345",
    "posts_per_day": 2,
    "is_active": true
  }
```

Important: kawn_community_id is the real community ID from your Kawn database. The engine needs it to know where to publish.

- PUT /api/communities/{engine_id} when community details change
- Set is_active to false when a community is disabled

Step 2 - Build ONE API endpoint on the Kawn backend (main work)

The Content Engine will call your API to create a post inside a community.

```
POST /api/v1/communities/{community_id}/posts
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json
```

Body the engine sends:

```
{
  "title": "France has announced its latest squad",
  "body": "Which player are you most excited to watch?",
  "post_type": "news_discussion",
  "tone": "passionate",
  "hashtags": ["france", "football"],
  "poll_options": null,
  "source_engine_post_id": "uuid-from-content-engine"
}
```

For polls, post_type will be poll and poll_options will be an array.

Your API must:

- Check the API key (Bearer token)
- Find the community by community_id
- Create a normal community post in Kawn DB
- Use a system/bot author (e.g. Kawn Community) - not a real user
- Return the new post ID as JSON: {"id": "your-kawn-post-id"}
- If source_engine_post_id already exists, return the existing post ID (no duplicates)

Step 3 - Give the Content Engine your API details

Set these environment variables on the Content Engine (Render):

- KAWN_APP_API_URL = your Kawn backend base URL
- KAWN_APP_API_KEY = the secret key your endpoint checks
- KAWN_AUTO_PUBLISH = true (posts go live automatically)

Flow: AI writes post -> safety check -> approved -> sent to Kawn API -> shows in app feed

What the Flutter app needs to do

- Keep loading the community feed from the Kawn backend as today
- Render poll posts if post_type is poll
- Optionally show a badge like Community host or AI post
- No direct calls to the Content Engine from the app

Daily posting

The Content Engine scheduler generates posts on a schedule. For about 2 posts per day per community, set posts_per_day to 2 when registering the community.

Registering communities with the right theme fields is enough to get themed daily content flowing. Stricter timing (e.g. exactly 8am and 8pm) is a Content Engine scheduler setting, not mobile work.

Checklist

- 1. Build POST /api/v1/communities/{id}/posts (Backend)
- 2. Add API key auth - Bearer token (Backend)
- 3. Store source_engine_post_id to avoid duplicates (Backend)
- 4. Sync communities to Content Engine with kawn_community_id (Backend)
- 5. Share API URL and key with Content Engine team (You / backend)
- 6. Show posts normally in Flutter feed (Mobile)
- 7. Support poll UI when post_type is poll (Mobile)

How to test

- Create a test community in Kawn
- Sync it to Content Engine with category, tags, and kawn_community_id
- In Content Engine admin, manually generate a post for that community
- Confirm it appears as Approved
- Publish it (or turn on KAWN_AUTO_PUBLISH)
- Check the post shows up in the Kawn app community feed

If publishing fails, check Kawn backend logs. The engine calls your API and expects a post ID back.

One sentence summary

Register each Kawn community in the Content Engine with its theme info and kawn_community_id, build one secure API endpoint that accepts posts into community feeds, and the engine will automatically publish themed daily content into the right communities.